LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 2

ANDROID LAYOUT WITH COMPOSE

Oleh:

Muhammad Naufal Khalish NIM. 2410817110004

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026

LEMBAR PENGESAHAN

LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE

MODUL 2

Laporan Praktikum Pemrograman Mobile Modul 2: Android Layout With Compose ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Muhammad Naufal Khalish
NIM : 2410817110004

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Erza Raihan NIM. 2310817210027	Irham Maulani Abdul Gani, S.Kom., M.Kom NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN ii
DAFTAR ISI iii
DAFTAR GAMBAR iv
DAFTAR TABEL v
SOAL 1 6
A. Source Code 8
B. Output Program 17
C. Pembahasan 19
SOAL 2 23
ADDITIONAL: ESSAY 25
DAFTAR PUSTAKA 28
TAUTAN GIT 29

DAFTAR GAMBAR

Gambar 1 Tampilan Awal Aplikasi 6
Gambar 2 Tampilan Pilihan Persentase Tip 7
Gambar 3 Tampilan Aplikasi Setelah Dijalankan 7
Gambar 4 Tampilan Aplikasi XML 17
Gambar 5 Tampilan Aplikasi Compose 18
Gambar 6 Output Program Test Case Data Classes menggunakan Kotlin Playground 27

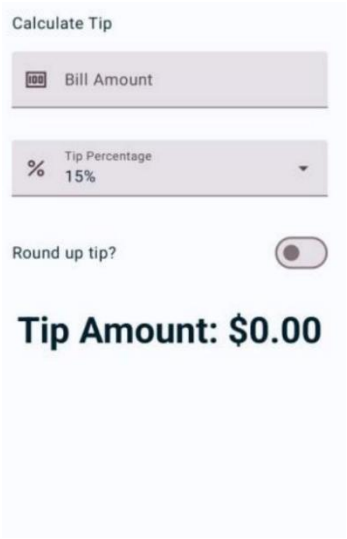
DAFTAR TABEL

Tabel 1 Source Code MainActivity.kt XML Soal 1 8
Tabel 2 Source Code activity_main.xml.....10
Tabel 3 Source Code MainActivity.kt Compose Soal 1 12

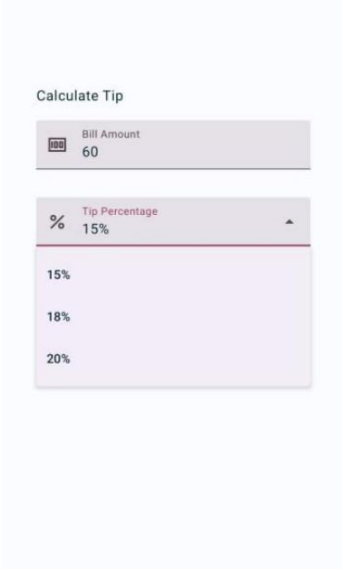
SOAL 1

Buatlah sebuah aplikasi kalkulator tip menggunakan XML dan Jetpack Compose yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:

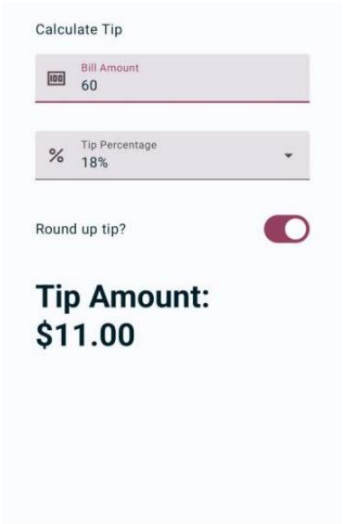
a. Input biaya layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.
b. Pilihan persentase tip: Pengguna dapat memilih persentase tip yang diinginkan.
c. Pengaturan pembulatan tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
d. Tampilan hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.



Gambar 1 Tampilan Awal Aplikasi



Gambar 2 Tampilan Pilihan Persentase Tip



Gambar 3 Tampilan Aplikasi Setelah Dijalankan

A. Source Code

MainActivity.kt - XML

Tabel 1 Source Code MainActivity.kt XML Soal 1

```
package com.example.calculatetipxml

import android.os.Bundle
import android.widget.ArrayAdapter
import androidx.appcompat.app.AppCompatActivity
import androidx.core.widget.doAfterTextChanged
import com.example.calculatetipxml.databinding.ActivityMainBinding
import java.text.NumberFormat

class MainActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMainBinding


    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)

        binding.tipResult.text = getString(R.string.tip_amount, "Rp0.00")

        val items = resources.getStringArray(R.array.tip_options_array)
        val adapter = ArrayAdapter(this, android.R.layout.simple_spinner_dropdown_item, items)
        binding.tipOptions.setAdapter(adapter)

        binding.costOfServiceText.doAfterTextChanged {
            calculateTip()
        }

        binding.tipOptions.setOnItemClickListener { _, _, _, _ ->
            calculateTip()
        }

        binding.roundUpSwitch.setOnCheckedChangeListener { _, _ ->
```

```
            calculateTip()
        }

    }

    private fun calculateTip(){
        val stringInTextField = binding.costOfServiceText.text.toString()
        val cost = stringInTextField.toDoubleOrNull()

        if(cost == null || cost == 0.0){
            binding.tipResult.text = getString(R.string.tip_amount, "Rp0.00")
            return
        }

        val selectedTipText = binding.tipOptions.text.toString().replace("%", "")
        val tipPercentage = selectedTipText.toDoubleOrNull() ?: 15.0

        var tip = tipPercentage / 100 * cost

        if(binding.roundUpSwitch.isChecked){
            tip = kotlin.math.ceil(tip)

        }

        val formattedTip = NumberFormat.getCurrencyInstance().format(tip)
        binding.tipResult.text = getString(R.string.tip_amount, formattedTip)
    }
}
```

activtiy_main.xml

Tabel 2 Source Code activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="20dp"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginLeft="16dp"
        android:text="@string/calculate_tip"
        android:textSize="16sp" />

    <com.google.android.material.textfield.TextInputLayout
        app:startIconDrawable="@drawable/money"
        android:id="@+id/cost_of_service_layout"
style="@style/Widget.Material3.TextInputLayout.FilledBox"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_margin="16dp"
        android:hint="@string/bill_amount">

<com.google.android.material.textfield.TextInputEditText


            android:id="@+id/cost_of_service_text"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="numberDecimal" />

    </com.google.android.material.textfield.TextInputLayout>

    <com.google.android.material.textfield.TextInputLayout
        app:startIconDrawable="@drawable/percent"

style="@style/Widget.Material3.TextInputLayout.FilledBox.Exp
osedDropdownMenu"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_margin="16dp"
```

```
        android:hint="@string/tip_percentage">

        <AutoCompleteTextView
            android:id="@+id/tip_options"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="none"
            android:text="@string/default_tip_percentage"/>

    </com.google.android.material.textfield.TextInputLayout>


    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:layout_margin="16dp"
        android:gravity="center_vertical">

        <TextView
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="@string/round_up_tip"
            android:textSize="16sp"/>

<com.google.android.material.switchmaterial.SwitchMaterial
            android:id="@+id/round_up_switch"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:checked="true"/>

    </LinearLayout>

    <TextView
        android:id="@+id/tip_result"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_margin="16dp"
        android:text="@string/tip_amount"
        android:textSize="32sp"
        android:textStyle="bold"
        android:textColor="@android:color/black"/>

</LinearLayout>
```

MainActivity.kt – Jetpack Compose

Tabel 3 Source Code MainActivity.kt Compose Soal 1

```
package com.example.calculatetipjetcompose

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.annotation.DrawableRes
import androidx.annotation.StringRes
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.res.stringArrayResource
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.ImeAction
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.calculatetipjetcompose.ui.theme.CalculateTipJetComposeTheme
import java.text.NumberFormat
import java.util.Locale

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        enableEdgeToEdge()
        super.onCreate(savedInstanceState)
        setContent {
            CalculateTipJetComposeTheme {
                Surface(modifier = Modifier.fillMaxSize()) {
                    CalculateTipApp()
                }
            }
        }
    }
}

```

```
@Composable
fun CalculateTipApp() {
    var amountInput by remember { mutableStateOf("") }
    var tipPercent by remember { mutableStateOf(15.0) }
    var roundUp by remember { mutableStateOf(false) }

    val amount = amountInput.toDoubleOrNull() ?: 0.0
    var tip = (tipPercent / 100) * amount
    if (roundUp) tip = kotlin.math.ceil(tip)


    val formattedTip = NumberFormat.getCurrencyInstance(Locale.US).format(tip)

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(40.dp)
            .verticalScroll(rememberScrollState()),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {

        Text(
            text = stringResource(R.string.calculate_tip),
            modifier = Modifier
                .align(Alignment.Start)
                .padding(bottom = 16.dp, top = 40.dp)
        )


        EditNumberField(
            label = R.string.bill_amount,
            leadingIcon = R.drawable.money,
            value = amountInput,
            onValueChanged = { amountInput = it },
            modifier = Modifier.padding(bottom = 32.dp).fillMaxWidth()
        )


        EditNumberSpinner(
            label = R.string.tip_percentage,
            leadingIcon = R.drawable.percent,
            selectedValue = tipPercent,
            onValueChanged = { tipPercent = it },
            modifier = Modifier.padding(bottom = 32.dp).fillMaxWidth()
        )
```

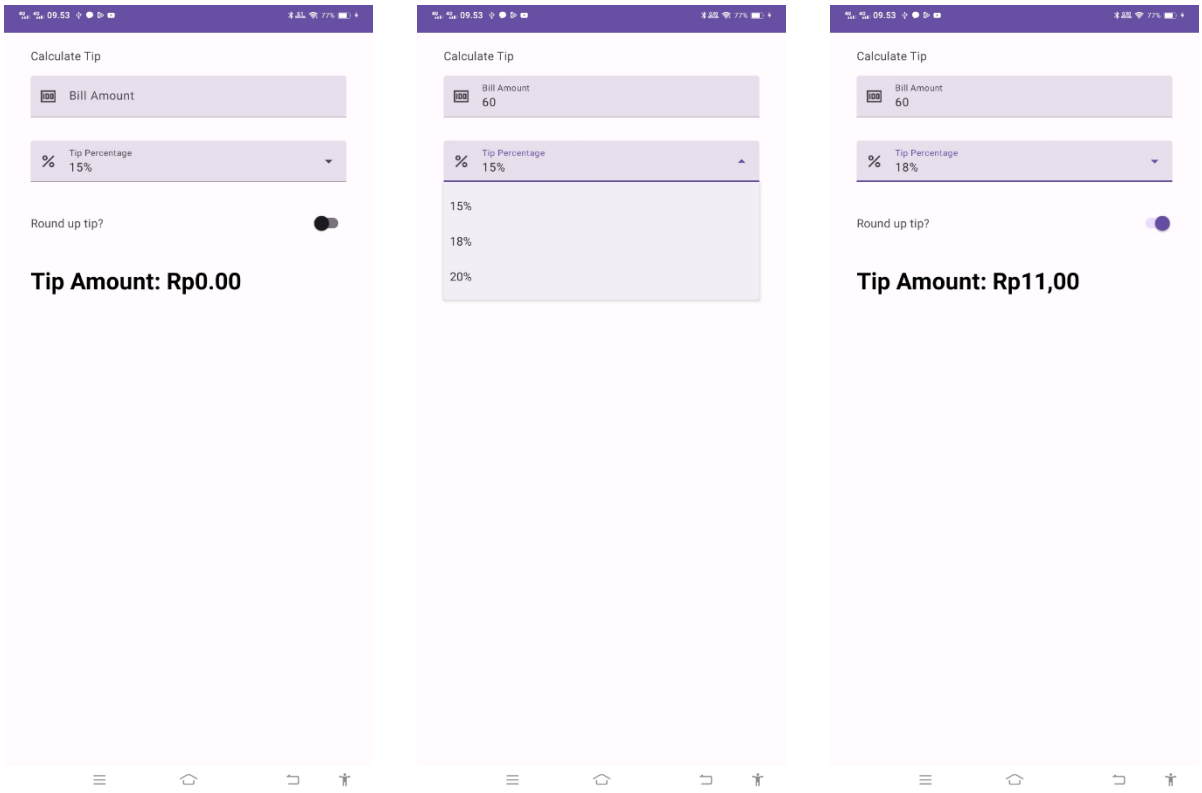
```


        RoundTheTipRow(
            roundUp = roundUp,
            onRoundUpChanged = { roundUp = it },
            modifier = Modifier.padding(bottom = 32.dp)
        )


        Text(
            text = stringResource(R.string.tip_amount, formattedTip),
            fontSize = 32.sp,
            fontWeight = FontWeight.Bold,
            modifier = Modifier.align(Alignment.Start)
        )
    }
}

@Composable
fun EditNumberField(
    @StringRes label: Int,
    @DrawableRes leadingIcon: Int,
    value: String,
    onValueChanged: (String) -> Unit,
    modifier: Modifier = Modifier
) {
    TextField(
        value = value,
        onValueChange = onValueChanged,
        leadingIcon = { Icon(painter = painterResource(id = leadingIcon), contentDescription = null) },
        label = { Text(stringResource(label)) },
        singleLine = true,
        keyboardOptions = KeyboardOptions.Default.copy(
            keyboardType = KeyboardType.Number,
            imeAction = ImeAction.Next
        ),
        modifier = modifier
    )
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun EditNumberSpinner(
    @StringRes label: Int,
    @DrawableRes leadingIcon: Int,
    selectedValue: Double,
```

```
    onValueChanged: (Double) -> Unit,
    modifier: Modifier = Modifier
) {
    var expanded by remember { mutableStateOf(false) }
    val options = stringArrayResource(R.array.tip_options_array)

    ExposedDropdownMenuBox(
        expanded = expanded,
        onExpandedChange = { expanded = !expanded },
        modifier = modifier
    ) {
        TextField(
            modifier = Modifier.menuAnchor().fillMaxWidth(),
            readOnly = true,
            value = "${selectedValue.toInt()}%",
            onValueChange = {},
            label = { Text(stringResource(label)) },
            leadingIcon = { Icon(painter = painterResource(id = leadingIcon), contentDescription = null) },
            trailingIcon = { ExposedDropdownMenuDefaults.TrailingIcon(expanded = expanded) },
            colors = ExposedDropdownMenuDefaults.textFieldColors(),
        )
        ExposedDropdownMenu(
            expanded = expanded,
            onDismissRequest = { expanded = false }
        ) {
            options.forEach { optionText ->
                DropdownMenuItem(
                    text = { Text(optionText) },
                    onClick = {
                        val value = optionText.replace("%", "").toDoubleOrNull() ?: 15.0
                        onValueChanged(value)
                        expanded = false
                    }
                )
            }
        }
    }
}

@Composable
fun RoundTheTipRow(
```

```
    roundUp: Boolean,
    onRoundUpChanged: (Boolean) -> Unit,
    modifier: Modifier = Modifier
) {
    Row(
        modifier = modifier.fillMaxWidth(),
        verticalAlignment = Alignment.CenterVertically
    ) {
        Text(text = stringResource(R.string.round_up_tip))
        Switch(
            modifier = Modifier
                .fillMaxWidth()
                .wrapContentWidth(Alignment.End),
            checked = roundUp,
            onCheckedChange = onRoundUpChanged
        )
    }
}

@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    CalculateTipJetComposeTheme {
        CalculateTipApp()
    }
}
```

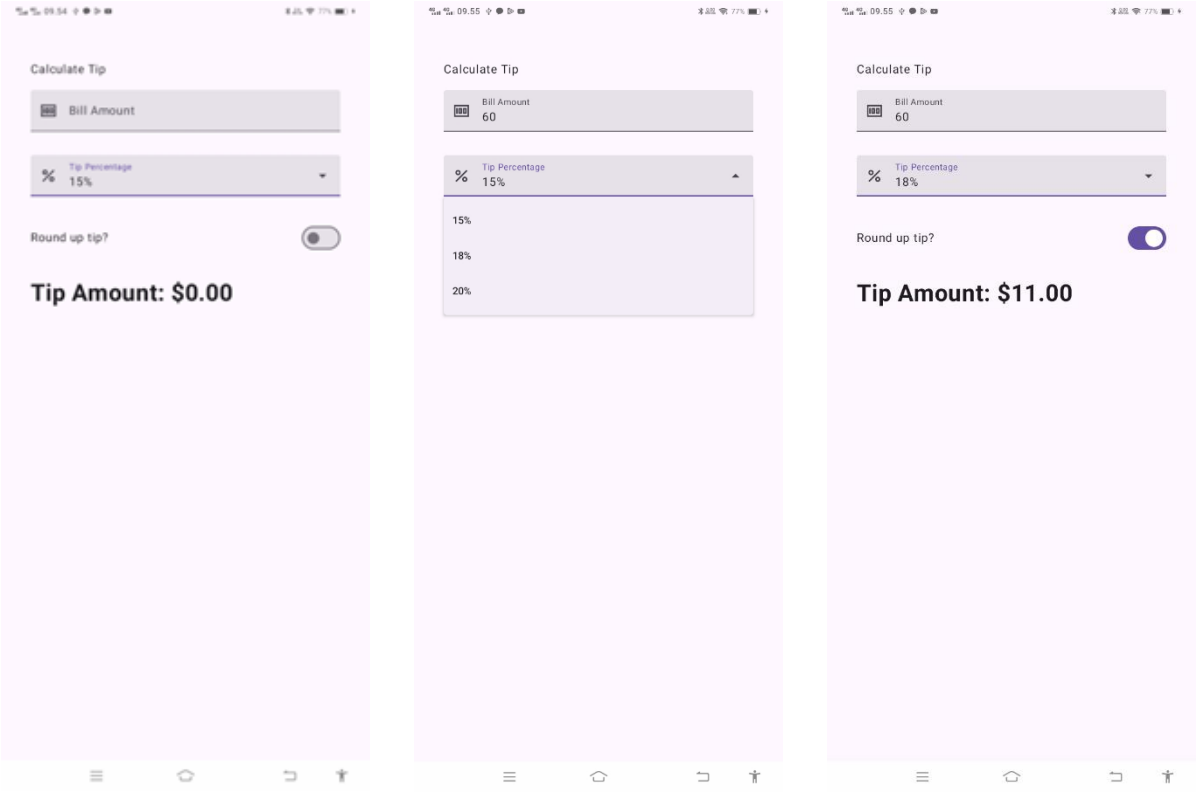
B. Output Program

XML



Gambar 4 Tampilan Aplikasi XML

Compose



Gambar 5 Tampilan Aplikasi Compose

C. Pembahasan

MainActivity.kt - XML

Pada MainActivity, sebagai kontrol dari tampilan yang dibuat pada file xml. Di dalamnya , menggunakan View Binding yang ditunjukkan pada variabel 'binding' yang menghubungkan xml dengan file ini tanpa harus menggunakan 'findViewId' pada xml. variabel ini diinisialiassi di dalam method onCreate menggunakan' ActivityMainBinding.inflate(layoutInflater)' yang tujuannya untuk membuat tampilan dari file xml, kemudian ditampilaknke layar menggunakan setCOntentView(binding.root) dimana root ini mengacu layout terluar dari xml. Selanjutnya, pada saat aplikasi dijalankan, nilai awal untuk hasil tip langsung ditampilkan dengan nilai "Rp0.00" menggunakan 'binding.tipResult.text'. kemudian aplikasi mengambil data pilihan persentae tip dari resource string-array yang dideifinisikan pada arrays.xml llau data tersebut dimasukkan ke dalam 'ArrayAdapter'. Adapter ini fungsinya sebagai penghubung antara data dan komponen dropdown 'AutoCompleteTextView' sehingga daftar persentase dapat ditampilkan dan dipilih .

Pada bagian interaksi pengguna, terdapat beberapa listener yang dipasang berdasarkan konponen input. 'doAfterTextChanged' digunakan pada field input biaya layanan untuk mendeteksi setiap perubahan teks yang diketik. Kedua, 'setOnItemClickListener' digunakan pada dropdown pilihan persentase tip untuk mendeteksi saat memilih salah satu opsi. Ketiga, 'setOnCheckedChangeListener' digunakan pada switch pembulatan untuk mendeteksi perubahan kondisi aktif atau tidaknya pembulatan. Ketiga listener ini memiliki fungsi yang sama, yaitu memanggil method 'calculateTip()' setiap kali terjadi perubahan, sehingga aplikasi dapat melakukan perhitungan secara otomatis dan real-time. Method 'calculateTip()' merupakan inti logika dari aplikasi yang bertugas menghitung nilai tip. Di dalamnya, pertama-tama diambil nilai input dari pengguna dalam bentuk string, kemudian dikonversi menjadi tipe data numerik (Double) menggunakan 'toDoubleOrNull()' untuk menghindari error jika input tidak valid. Jika nilai yang diperoleh bernilai null atau nol, maka aplikasi akan langsung menampilkan "Rp0.00" dan menghentikan proses perhitungan. Selanjutnya, aplikasi mengambil nilai persentase tip dari dropdown, menghapus simbol persen (%), lalu mengubahnya menjadi angka dengan nilai default 15% jika terjadi kesalahan konversi. Setelah itu, dilakukan proses perhitungan tip dengan mengalikan persentase terhadap total biaya layanan. Jika fitur pembulatan diaktifkan melalui switch, maka hasil tip akan dibulatkan ke atas menggunakan fungsi ceil(). Hasil perhitungan tersebut kemudian diformat ke dalam bentuk mata uang menggunakan 'NumberFormat.getCurrencyInstance()'. Terakhir, nilai tip yang telah diformat ditampilkan kembali ke layar melalui komponen TextView.

Activity_main.xml

Pada bagian Linear Layout yang gunanya membungkus komponen komponen di dalamnya yaotu ‘android:layout_width=”match_parent” ‘ dan ‘android:layout_width=”match_parent” ‘ yang digunakan mengatur lebar dan tinggi mengikuti layar pengguna, kemudian dengan orientasi vertikal dan elemen elemen di dalamnya akan berderet menurun ke bawah, kemudian padding sebesar 20 dp dan file xml ini dikontrol oleh file kotlin bernama MainActivity.

TextView dibawahnya terdapat komponen di dalamnya yang mengatur ukuran komponen sesuai dengan konten teks dengan padding 16 dp dan teks nya diambil dari package string dengan name “calculate_tip” dengan ukuran teks sebesar 16 sp

Lalu TextInputLayout yang fungsinya sebagai input, terdapat konfigurasi ikon di sisi awal dengan mengambil asset dari drawable bernama “money” dengan id “cost_of_service_layout” yang digunakan untuk akses kode kotlin, menggunakan gaya tampilan dari Material3FilledBox dengan ukuran lebar memenuhi layar dan tinggi sesuai isi konten. Dengan margin sebesar 16 dp, serta menampilkan teks petunjuk (*hint*) yang diambil dari string bernama “bill_amount”.

Pada bagian TextInputEditText yang berada di dalam TextInputLayout, terdapat identitas ID dengan nama "cost_of_service_text", memiliki ukuran lebar yang memenuhi bingkainya dan tinggi sesuai konten, serta menggunakan tipe input “numberDecimal” yang berfungsi untuk membatasi agar hanya bisa memasukkan angka desimal melalui keyboard.

Pada bagian TextInputLayout untuk pilihan tip, terdapat ikon di sisi awal yang mengambil gambar dari drawable bernama "percent", menggunakan gaya tampilan ExposedDropdownMenu agar terlihat seperti menu pilihan, dengan lebar memenuhi layar dan tinggi sesuai konten, memiliki jarak luar (margin) sebesar 16 dp, serta menampilkan teks petunjuk dari string bernama "tip_percentage".

Pada bagian AutoCompleteTextView yang berada di dalam TextInputLayout kedua, terdapat identitas ID dengan nama "tip_options", memiliki ukuran lebar yang memenuhi bingkainya dan tinggi sesuai konten, menggunakan tipe input "none" agar tidak dapat diketik manual melainkan hanya bisa memilih opsi, serta menampilkan teks default yang diambil dari string bernama "default_tip_percentage".

Pada bagian LinearLayout yang membungkus pengaturan pembulatan, terdapat ukuran lebar yang memenuhi layar dan tinggi sesuai konten (wrap_content), menggunakan orientasi horizontal agar elemen

di dalamnya berderet ke samping, memiliki jarak luar (margin) sebesar 16 dp, serta dilengkapi pengaturan gravity center_vertical agar teks dan tombol switch di dalamnya sejajar lurus di tengah secara vertikal.

Pada bagian TextView yang berada di dalam LinearLayout horizontal, terdapat ukuran lebar 0 dp namun menggunakan 'layout_weight="1" yang gunanya untuk mengambil seluruh sisa ruang kosong sehingga mendorong komponen di sebelahnya ke ujung kanan, memiliki tinggi sesuai konten, menampilkan teks dari string bernama "round_up_tip", serta memiliki ukuran teks sebesar 16 sp.

Pada bagian SwitchMaterial yang berfungsi sebagai tombol pilihan, terdapat identitas ID dengan nama "round_up_switch", memiliki ukuran lebar dan tinggi sesuai dengan bentuk tombolnya (wrap_content), serta memiliki pengaturan checked="true" yang gunanya untuk membuat status tombol ini sudah dalam keadaan aktif (tercentang) secara otomatis saat aplikasi dijalankan.

Pada bagian TextView yang berada di posisi paling bawah, terdapat identitas ID dengan nama "tip_result", memiliki ukuran lebar memenuhi layar dan tinggi sesuai konten, memiliki jarak luar (margin) sebesar 16 dp, menampilkan teks awal dari string bernama "tip_amount", serta menggunakan ukuran teks yang sangat besar yaitu 32 sp dengan gaya teks tebal (bold) dan warna hitam

MainActivity.kt – Compose

Pada bagian '@Composable fun CalculateTipApp()' , fungsi ini berperan sebagai pusat dari seluruh tampilan dan logika aplikasi dalam Jetpack Compose. Berbeda dengan pendekatan XML yang memisahkan tampilan dan logika, pada Compose keduanya ditulis dalam satu fungsi secara deklaratif. Di dalam fungsi ini, pertama-tama dideklarasikan beberapa state menggunakan remember dan 'mutableStateOf' , yaitu 'amountInput' untuk menyimpan input dalam bentuk teks, 'tipPercent' untuk menyimpan persentase tip, serta 'roundUp' untuk menentukan apakah hasil tip perlu dibulatkan. State ini bersifat reaktif, sehingga setiap kali nilainya berubah, tampilan akan otomatis diperbarui tanpa perlu pemanggilan fungsi manual seperti pada pendekatan sebelumnya.

Selanjutnya, nilai input yang berupa string dikonversi menjadi angka (Double) menggunakan 'toDoubleOrNull()' untuk menghindari error jika input tidak valid. Nilai tip kemudian dihitung berdasarkan persentase terhadap jumlah biaya, dan apabila opsi pembulatan aktif, maka hasilnya dibulatkan ke atas menggunakan fungsi 'ceil()'. Hasil perhitungan ini kemudian diformat menjadi bentuk mata uang menggunakan 'NumberFormat' dengan locale tertentu

Untuk bagian tampilan, digunakan komponen 'Column' sebagai wadah utama dengan susunan vertikal,

yang diatur menggunakan ‘Modifier’ seperti ‘fillMaxSize’ , ‘padding’. Di dalam Column, terdapat beberapa komponen seperti ‘Text’ untuk judul, ‘EditNumberField’ untuk input jumlah biaya, ‘EditNumberSpinner’ untuk memilih persentase tip, serta ‘RoundTheTipRow’ untuk mengaktifkan opsi pembulatan. Setiap komponen ini menerima state dan callback sebagai parameter, sehingga ketika pengguna melakukan interaksi, nilai state akan berubah dan secara otomatis memicu recomposition (pembaruan tampilan).

Terakhir, hasil tip yang telah diformat ditampilkan menggunakan komponen Text dengan ukuran dan ketebalan font yang lebih besar untuk menonjolkan hasil perhitungan.

SOAL 2

Jelaskan perbedaan dari implementasi XML dan Jetpack Compose beserta kelebihan dan kekurangan dari masing-masing implementasi.

Struktur Hierarki Layout

XML mengimplementasikan layout sebagai struktur tree yang statis dan permanen di memori. Setiap elemen seperti 'Linear Layout' adalah objek view yang harus diinstansiasi satu per satu, sehinggga perubahan diperlukan pemanggilan secara manual dari kode program. Sebaliknya Jetpack Compose menggunakan fungsi komposisi yang bersifat deklaratif, dimana layout seperti 'Column' dan 'Row' hanya instruksi yang dijalankan ulang secara otomatis saat terjadi perubahan data.

XML unggul dalam pemisahan desain dan logika serta pratinjau yang stabil, namun kekurangannya adalah performa yang dikorbankan jika hierarki berbentuk nested terlalu dalam. Sementara Compose unggul dalam struktur layout dengan lebih datar dan ringan, tetapi kekurangannya adalah risiko render ulang yang berlebihan jika logika data tidak dikelola dengan baik

Penempatan Posisi Elemen

XML sangat bergantung pada atribut yang dituliskan dalam tag seperti 'layout_gravity' atau koordinat relatif pada 'ConstraintLayout'. Semua aturan posisi ditentukan sebelum aplikasi dijalnakan dan sulit diubah secara mendadak. Di sisi lain, Jetpack Compose menggunakan sistem Modifier yang dirantai untuk menentukan posisi, ukuran, dan perilaku elemen.

XML memiliki kelebihan pada kemudahan positioning melalui fitur drag-and-drop di editor, namun kekurangannya adalah saat ingin membuat tata letak yang sangat dinamis. Compose memiliki fleksibiltas tinggi karena tata letak bisa berubah secara instan menggunakan logika kode, namun kekurangannya adalah urutan penulisan Modifier yang sensitif yang dimana perbedaan urutan akan menghasilkan hasil tampilan yang berbeda juga

List Layouting

XML menggunakan 'RecyclerView' yang dimana diharuskan membuat banyak komponen tambahan untuk menghubungkan data dengan layout. Sedangkan Jetpack Compose menggunakan LazyColumn atau LazyRow yang jauh lebih ringkas, dimana item daftar didefinsikan langsung di dalam fungsi layout.
Kelebihan XML terletak pada efisiensi memori untuk jumlah item yang kompleks, namun kekurangannya

terjadi pada boilerplate yang harus ditulis. Kelebihan Compose adalah kecepatan pengembangan karena kodenya yang easy to read, namun kekurangan pada performa scrolling yang kurang mulus jika fungsi di dalam tiap item daftar terlalu berat

ADDITIONAL:

ESSAY

1. Apa saja fitur yang menjadi daya jual pada Kotlin? (contoh: null safety, chain, dll)**
2. berikan 1 test case terkait penggunaan fitur unggulan yang ada pada kotlin (contoh: penggunaan null safety)

Soal 1

1. Coroutines

Salah satu fitur paling menonjol yang menjadi daya tarik Kotlin adalah coroutines sebuah mekanisme pemrograman asinkron yang dirancang untuk menyelesaikan masalah klasik dalam pengembangan perangkat lunak modern [1]. Coroutines di Kotlin menggunakan satu kata kunci sederhana, yaitu suspend, yang memungkinkan sebuah fungsi untuk dijeda dan dilanjutkan kembali tanpa memblokir thread yang menjalankannya. Pendekatan ini secara fundamental berbeda dari pemrograman multi-thread konvensional yang boros memori, maupun dari pola callback yang menghasilkan kode berlapis dan sulit dibaca.

Daya tarik utama coroutines terletak pada kemampuannya membuat kode asinkron terlihat dan terasa seperti kode sekuensial biasa. Developer tidak perlu lagi menulis callback bertingkat atau memahami kompleksitas reactive programming

2. Null Safety

Null safety merupakan fitur penting dalam kotlin yang mengatasi masalah klasik berupa ‘null reference exception’ , atau pada bahasa java dikenal dengan ‘NullPointerException’. Dalam bahasa Java, mencoba atribut dari objek yang bernilai ‘null’ seperti memanggil ‘.length() pada string yang kosong akan menyebabkan program error. Oleh karena itu Kotlin di desain dengan sistem null safety yang lebih ketat untuk menghindari kegagalan program tersebut. Kotlin membedakan antara tipe data yang nullable dan tidak dengan menggunakan operator safecall “?”. Kotlin tidak akan memaksakan pemanggilan fungsi jika objeknya bernilai null [2]. Alih-alih menghentikan program dengan eksepsi, Kotlin hanya akan mengembalikan nilai null sebagai output

3. **Data Class**

Data Class dalam kotlin adalah class khusus yang spesifik sebagai wadah data atau status suatu objek. Ini berbeda dengan class pada biasanya yang memerlukan penulisan kode yang panjang. Kotlin secara otomatis menghasilkan fungsi-fungsi standar seperti equals(), hashCode(), toString(), serta fungsi copy() untuk menyalin objek dengan modifikasi tertentu.[3]

Soal 2

Test case penggunaan data classes

```
1   package com.example.myapplication
2
3   data class User(val id: Int, val name: String)
4
5   fun main(){
6       println("Test Fungsi Data Classes")
7       toString()
8       equal()
9       hashCode()
10      copy()
11      destructuring()
12  }
13
14  fun toString(){
15      val user = User(1, "Zaki")
16      println(user.toString())
17  }
18
19  fun equal(){
20      val userA = User(1, "Zaki")
21      val userB = User(1, "Zaki")
22      println(userA == userB)
23  }
24
25  fun hashCode(){
26      val userA = User(1, "Zaki")
27      val userB = User(1, "Khalish")
28      println(userA.hashCode() == userB.hashCode())
29  }
30
31  fun copy(){
32      val userA = User(1, "Zaki")
33      val userB = userA.copy(name = "Khalish")
34      println(userB)
35  }
365
37
```

```
38  fun destructuring(){
39      val user = User(1, "Zaki")
40      val (id, name) = user
41      println("$id, $name")
42  }
```

Output Program

```
Test Fungsi Data Classes
User(id=1, name=Zaki)
true
false
User(id=1, name=Khalish)
1, Zaki
```

Gambar 6 Output Program Test Case Data Classes menggunakan Kotlin Playground

DAFTAR PUSTAKA

[1] R. Elizarov, *Kotlin Coroutines : Design and Implementation*, vol. 1, no. 1. Association for Computing Machinery. doi: 10.1145/3486607.3486751.

[2] S. S. Sharma, Akshit Kumar Prasad, "Kotlin vs Java: Its Practical Implications," in *Proceedings of the KILBY 100 7th International Conference on Computing Sciences 2023 (ICCS 2023)*, SSRN, 2023. doi: 10.2139/ssrn.4492426.

[3] J. S.r.o., "Data classes," Kotlin Help. Accessed: Apr. 19, 2026. [Online]. Available: https://kotlinlang.org/docs/data-classes.html

TAUTAN GIT

Berikut adalah tautan untuk repositori GitHub yang telah dibuat

Link Github